PPE Compliance Gate: Hardware Build Guide (Beginner, step by step)

Who this is for: someone who has never wired an Arduino. **What you will build:** the complete hardware of the PPE gate: an Arduino Uno controlling a servo barrier, a green LED, a red LED, a buzzer, an I2C LCD and an ultrasonic distance sensor, all mounted on a small cardboard gate.

How to use it: do the stages in order. After every stage there is a test. **Do not move to the next stage until the test works.** If something fails, you then know the fault is in the part you just added.

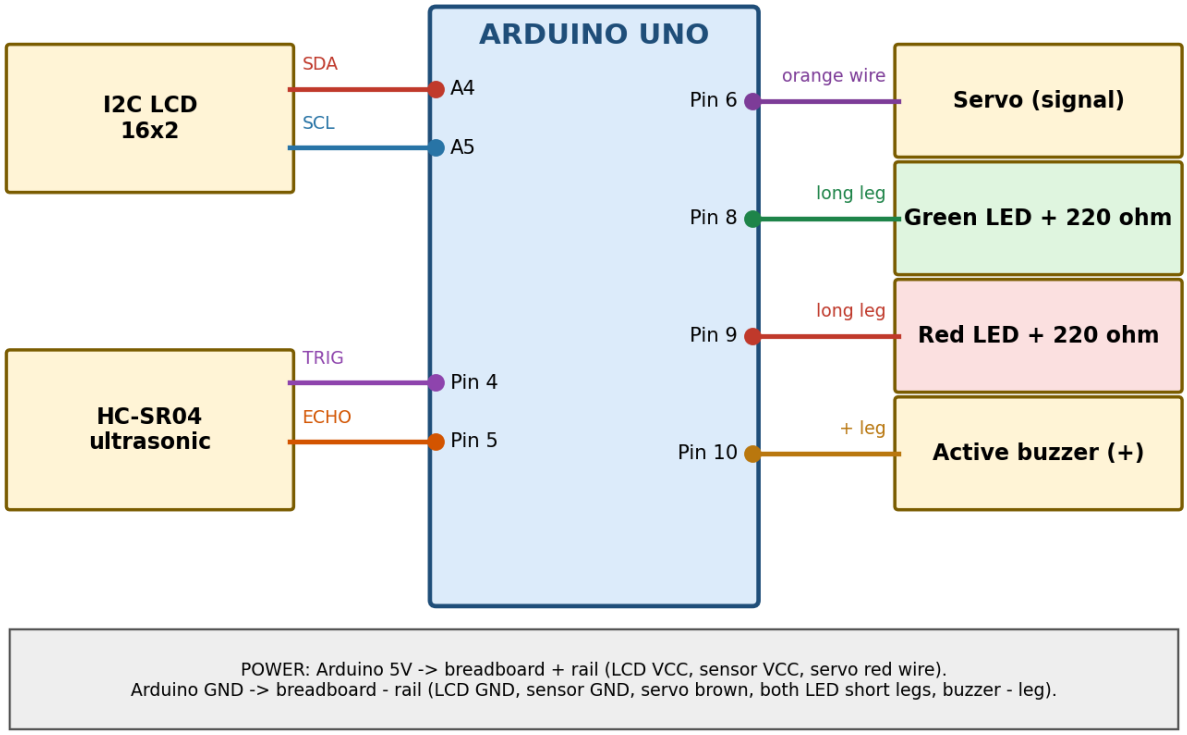
Not covered here: the AI, Python and Telegram software. This guide ends with a finished, tested gate that is ready for the software.

Estimated time: about 6-9 hours in total (wiring and testing 3-5 hours, gate building 3-4 hours).

---

1. What you are building

Figure 1: Connection map (signal wires)



In the finished project the laptop will send one-letter commands over the USB cable, and the Arduino will switch the outputs. The Arduino also reads the distance sensor and sends the distance back to the laptop. For this hardware guide, you will test everything by typing the commands yourself.

Part	What it does in the project
Arduino Uno	The small controller. Everything connects to it.
Green LED	Lights up when access is granted.
Red LED	Lights up when access is denied.
Active buzzer	Beeps when access is denied.
SG90 servo	Swings the barrier arm between closed (0 degrees) and open (90 degrees).
16x2 I2C LCD	Shows messages such as "ACCESS GRANTED" or "Missing: HELMET".
HC-SR04 ultrasonic sensor	Measures the distance to a person, so the gate wakes up only when someone steps close.

Part	What it does in the project
470 uF capacitor	Smooths out power spikes when the servo moves, so the Arduino does not reset.

2. Parts and tools checklist

Tick each item before you start.

Item	Qty	How to recognise it
Arduino Uno (clone is fine) + USB cable (Type-B, the squarish plug)	1	A blue or green board with a USB socket and rows of black sockets
Half-size breadboard	1	White plastic board full of small holes, with two long lines along each edge
Green LED and red LED (5 mm)	1 each + 1 spare	Small coloured domes, with one long leg and one short leg
220-ohm resistors	2 + 2 spare	Small cylinders with colour bands red, red, brown, then gold
Active buzzer (5V)	1	Black cylinder with a sealed top and a + mark. A passive buzzer shows a green circuit board underneath, so avoid that kind.
SG90 micro servo with horn and screws	1	Small blue or black box with 3 wires and white plastic arms in a bag
16x2 LCD with I2C backpack	1	Green LCD with a small board soldered on the back, with only 4 pins: GND, VCC, SDA, SCL
HC-SR04 ultrasonic sensor	1	Blue board with two silver cylinders ("eyes") and 4 pins: VCC, Trig, Echo, GND
470 uF electrolytic capacitor (16V or higher)	1	Small can with a stripe and a minus sign on one side
Jumper wires	see note	Male-male (pins on both ends), male-female (a pin and a socket), female-female

How many wires? You need about **14 male-to-female** wires (4 for the LCD, 4 for the sensor, 4 for extending the LED legs, 2 for the buzzer) plus about **11 male-to-male** wires (8 on the breadboard and 3 for the servo plug). A mixed pack of about 40 is usually enough, but if you run short, buy an extra pack of 20 male-to-female wires (about 30-50 rupees).

For the gate: a cardboard box (about 30 x 20 x 15 cm, such as a shoe box), ice-cream sticks or a ruler for the arm, double-sided tape, hot glue gun or strong glue, electrical tape, a sharp cutter or scissors, a pencil, a ruler, red and white tape or paper for the arm stripes.

3. Five basics you must know first

1. The breadboard. It lets you connect parts without soldering. The holes are joined in a pattern:

- Along the board, the numbered **strips** (columns, numbered 1 to 30) each have two groups of 5 holes: **a-b-c-d-e** and **f-g-h-i-j**. The 5 holes in one group of one strip are all connected to each other. The gap in the middle separates the two groups.
- The long lines along the top and bottom edges are the **power rails**. All holes in one rail are connected the whole length of the board. Use the top rails: the one marked **+** (usually red) will be 5V, and the one marked **-** (usually blue) will be ground.
- If two wires or legs share a connected group of holes, they are electrically joined.

2. Power and ground. Every part needs a power wire (5V) and a return wire (GND, also called ground). We connect Arduino 5V to the **+ rail** and Arduino GND to the **- rail**, and then take power and ground for every part from those rails.

3. LED polarity. An LED works only one way round. The **long leg is positive (+)** and goes towards the Arduino pin. The **short leg is negative (-)** and goes towards ground. An LED must **always** have a

220-ohm resistor in series, or it burns out.

4. Where the Arduino pins are. The top row of black sockets is the digital pins, marked 0 to 13 (use 4, 5, 6, 8, 9 and 10). The bottom row has the power pins (5V, GND) on the left and the analog pins A0 to A5 on the right (use A4 and A5). There are also GND sockets next to pin 13.

5. Safety rules.

- **Always unplug the USB cable before changing any wiring.**
- Never connect 5V straight to GND with a wire (a short circuit).
- Plug the capacitor in the right way round: the long leg to + and the striped leg to -.
- If anything smells hot or gets very warm, unplug immediately.

Habit that will save you hours: after wiring each stage, take a clear photo of the breadboard before you plug in the USB cable. If something fails, the photo lets you compare with this guide (or send it to me).

Stage 0: Software setup and board test (30-45 minutes)

1. Install the **Arduino IDE** (free, from arduino.cc) on your laptop.
2. Plug the Arduino into the laptop with the USB cable. A green light on the board should come on.
3. If Windows does not recognise a clone board, search for "CH340 driver", install it, and plug the board in again.
4. In the IDE choose **Tools > Board > Arduino Uno**. Then choose **Tools > Port** and pick the port that appears when you plug the board in (for example COM3). If you are unsure, unplug the board, look at the list, plug it in again, and pick the new entry.
5. Open **File > Examples > 01.Basics > Blink** and click the **Upload** button (the right arrow at the top).

Test: the small LED marked "L" on the Arduino blinks once a second. If yes, your laptop, cable and board all work.

If the upload fails: check the board and port in the Tools menu, try another USB cable or port, and install the CH340 driver.

Stage 1: Set up the power rails and the LEDs (45-60 minutes)

Unplug USB first. All the wire positions below use breadboard strip numbers. Use the top rails for + and -.

Step 1: power rails.

1. Use a male-to-male jumper from Arduino **GND** (bottom power header) to the **- rail** of the breadboard.
2. Use another jumper from Arduino **5V** to the **+ rail**.

Step 2: green LED.

1. Jumper from Arduino **pin 8** to **strip 5** (hole 5a).
2. Resistor (220 ohm) from **5c** to **9c**. It has no direction, so it can go either way round.
3. Green LED **long leg in 9d**, **short leg in 10d**.
4. Jumper from **10a** to the **- rail**.

Step 3: red LED.

1. Jumper from Arduino **pin 9** to **strip 13** (hole 13a).
2. Resistor from **13c** to **17c**.
3. Red LED **long leg in 17d**, **short leg in 18d**.
4. Jumper from **18a** to the **- rail**.

Test code. Plug in USB, open a new sketch (File > New), paste this, and click Upload:

```
void setup() {
  pinMode(8, OUTPUT);
  pinMode(9, OUTPUT);
```

```
}

void loop() {
  digitalWrite(8, HIGH);   // green on
  delay(1000);
  digitalWrite(8, LOW);
  digitalWrite(9, HIGH);   // red on
  delay(1000);
  digitalWrite(9, LOW);
}
```

Test: the green LED lights for one second, then the red LED for one second, repeating.

Problem	Fix
An LED never lights	It is the wrong way round. Take it out and turn it 180 degrees.
Neither LED lights	Check that the - rail is connected to Arduino GND, and that the resistors are in their strips.
The colours are swapped	The pin 8 and pin 9 jumpers are in the wrong strips.
An LED is very dim	Check the resistor is 220 ohm (red, red, brown) and not a much larger value.

Later (Stage 7) you will move each LED to the front panel of the gate. For that, extend its legs with male-to-female wires (explained in Stage 7), so you can leave the resistor part as built here.

Stage 2: Buzzer (20-30 minutes)

Unplug USB first.

1. Place the buzzer so its **long leg (the + leg)** is in **strip 22** (hole 22d) and its other leg is in **strip 25** (hole 25d). If the legs do not reach, bend them slightly outwards.
2. Jumper from **22a** to Arduino **pin 10**.
3. Jumper from **25a** to the **- rail**.

Quick check that it is an active buzzer: touch the buzzer's two legs briefly to the + rail and - rail (with USB plugged in). An active buzzer beeps by itself. A passive one stays silent, and if so you need to swap it for an active one.

Test code:

```
void setup() {
  pinMode(10, OUTPUT);
}

void loop() {
  digitalWrite(10, HIGH);
  delay(200);
  digitalWrite(10, LOW);
  delay(800);
}
```

Test: a short beep every second.

Problem	Fix
No sound	Swap the buzzer's legs (the + leg must face pin 10), and check the - rail connection.
A continuous tone at the start	Normal if you touched it to the rails. Remove it and re-test with the code.
Still no sound	The buzzer is probably passive. Exchange it for an active one.

Stage 3: Servo and capacitor (30-45 minutes)

Unplug USB first. The servo has three wires:

Servo wire	Goes to
Brown (or black)	**- rail** (ground)
Red	**+ rail** (5V)
Orange (or yellow)	Arduino **pin 6**

How to connect: the servo ends in a small plastic plug with three **sockets**. Push the pin end of a **male-to-male** wire into each socket, matching the colour order of the servo wires (brown, red, orange). Then put the other ends in the right places: the brown one to the - rail, the red one to the + rail, and the orange one straight into the Arduino pin 6 socket. Wrap tape around the joins so they stay on.

Capacitor: place the **470 uF capacitor across the rails**: the **long leg in the + rail** and the **striped leg (marked with a minus sign) in the - rail**, close to where the servo wires connect.

Test code:

```
#include <Servo.h>
Servo gate;

void setup() {
  gate.attach(6);
}

void loop() {
  gate.write(0);     // closed position
  delay(1500);
  gate.write(90);    // open position
  delay(1500);
}
```

Test: the servo horn swings back and forth, 90 degrees, with a pause at each end. Do not attach the long arm yet.

Problem	Fix
The servo does not move	Check the orange wire is in pin 6, and the red and brown wires are on the correct rails.
The servo twitches and the Arduino restarts (the green light flickers)	Check that the capacitor is the right way round and close to the servo. If it still resets, power the servo from a separate 5V supply sharing GND.
The servo hums at the ends	Normal for a small hum. If the arm is blocked by something, free it.

Stage 4: I2C LCD (45-60 minutes)

Step 1: install the library. In the Arduino IDE open **Sketch > Include Library > Manage Libraries**, search for **LiquidCrystal I2C** (by Frank de Brabander), and click **Install**.

Step 2: wire it (USB unplugged). The LCD backpack has 4 pins. Check the printed labels on it, because the order can differ.

LCD pin	Goes to
GND	**- rail**
VCC	**+ rail**
SDA	Arduino **A4**
SCL	Arduino **A5**

Use male-to-female wires: the female end goes on the LCD pin and the male end goes into the rail or the Arduino socket.

Step 3: find the address. An I2C LCD has an "address", usually 0x27 or 0x3F. Upload this scanner, open **Tools > Serial Monitor** (set the speed to **9600** at the bottom right), and read the result:

```
#include <Wire.h>
void setup() {
  Serial.begin(9600);
  Wire.begin();
  for (byte a = 1; a < 127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) {
      Serial.print("Found device at 0x");
      Serial.println(a, HEX);
    }
  }
}
```

```
void loop() {}
```

It should print something like `Found device at 0x27`. Use that number in the next code (if it says 3F, replace 0x27 with 0x3F).

Step 4: show text.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);   // use your address here

void setup() {
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("PPE Gate");
  lcd.setCursor(0, 1);
  lcd.print("Hello!");
}

void loop() {}
```

Test: the LCD shows "PPE Gate" on the first line and "Hello!" on the second.

Problem	Fix
The backlight is on but there is no text, or only blocks on the top row	Turn the small blue screw (the contrast knob) on the back of the LCD backpack slowly with a small screwdriver until text appears.
The scanner finds nothing	Check SDA is on A4 and SCL on A5 (not swapped), and GND and VCC are connected.
A compile error about "LiquidCrystal_I2C"	The library is not installed. Repeat Step 1.
The text is garbled	The wrong address or loose wires. Re-check and re-upload.

Stage 5: Ultrasonic sensor (30-45 minutes)

Unplug USB first. The HC-SR04 has 4 pins, labelled on the board:

Sensor pin	Goes to
VCC	**+ rail**
Trig	Arduino **pin 4**
Echo	Arduino **pin 5**
GND	**- rail**

Use male-to-female wires, with the female ends on the sensor pins.

Test code:

```
const int TRIG = 4;
const int ECHO = 5;

void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
}

void loop() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long t = pulseIn(ECHO, HIGH, 30000);
  if (t == 0) {
    Serial.println("no echo");
  } else {
    Serial.print(t / 58);
    Serial.println(" cm");
  }
  delay(300);
}
```

Open **Tools > Serial Monitor** at **9600** speed.

Test: a number appears about three times a second. Hold your hand 20 cm in front of the sensor and then 50 cm: the number should change to about 20 and then about 50. Point it at a wall and check it roughly matches a ruler.

Problem	Fix
"no echo" every time	Check all four wires, especially VCC and GND, and that Trig and Echo are not swapped. Make sure nothing blocks the two silver "eyes".
The numbers jump around	Aim the sensor straight at a flat surface. Soft cloth at an angle gives unreliable readings.
It always reads very small (2-3 cm)	Something is touching or blocking the sensor, or the Trig and Echo wires are loose.

Stage 6: Combine everything and test with commands (45-60 minutes)

Now all parts are wired. Upload the final sketch below. It is the same one the software will use, so keep it.

How it works: the Arduino waits for a one-letter command from the Serial Monitor (later, from the laptop program) and reports the distance five times a second.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

const int GREEN_LED = 8;
const int RED_LED   = 9;
const int BUZZER    = 10;
const int SERVO_PIN = 6;
const int TRIG_PIN  = 4;
const int ECHO_PIN  = 5;

const int GATE_CLOSED = 0;
const int GATE_OPEN   = 90;
const unsigned long PING_EVERY_MS = 200;

LiquidCrystal_I2C lcd(0x27, 16, 2);   // use your address (0x27 or 0x3F)
Servo gate;
unsigned long lastPing = 0;

void show(const char* l1, const char* l2) {
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print(l1);
  lcd.setCursor(0, 1); lcd.print(l2);
}

void beep(int times, int ms) {
  for (int i = 0; i < times; i++) {
    digitalWrite(BUZZER, HIGH); delay(ms);
    digitalWrite(BUZZER, LOW);  delay(ms);
  }
}

void setIdle() {
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);
  gate.write(GATE_CLOSED);
  show("PPE Gate", "Step closer");
}

void granted() {
  digitalWrite(RED_LED, LOW);
  digitalWrite(GREEN_LED, HIGH);
  gate.write(GATE_OPEN);
  show("ACCESS GRANTED", "Please enter");
  beep(1, 100);
}

void denied(const char* line2) {
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);
  gate.write(GATE_CLOSED);
```

```
  show("ACCESS DENIED", line2);
  beep(3, 150);
}

long readDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long d = pulseIn(ECHO_PIN, HIGH, 30000);
  if (d == 0) return 999;
  return d / 58;
}

void handleCommand(char c) {
  switch (c) {
    case 'C':
      digitalWrite(GREEN_LED, LOW);
      digitalWrite(RED_LED, LOW);
      show("Checking PPE", "Hold still...");
      break;
    case 'G': granted(); break;
    case 'R': denied("No helmet+vest"); break;
    case 'H': denied("Missing: HELMET"); break;
    case 'V': denied("Missing: VEST"); break;
    case 'I': setIdle(); break;
    default: break;
  }
}

void setup() {
  Serial.begin(9600);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  gate.attach(SERVO_PIN);
  lcd.init();
  lcd.backlight();
  setIdle();
}

void loop() {
  if (Serial.available() > 0) {
    handleCommand(Serial.read());
  }
  if (millis() - lastPing >= PING_EVERY_MS) {
    lastPing = millis();
    Serial.print("D:");
    Serial.println(readDistanceCm());
  }
}
```

Test it: open **Tools > Serial Monitor**, set the speed to **9600** and the line ending (dropdown at the bottom) to **Newline**. You will see lines like `D:87` appearing. Type each command in the box at the top and press Enter:

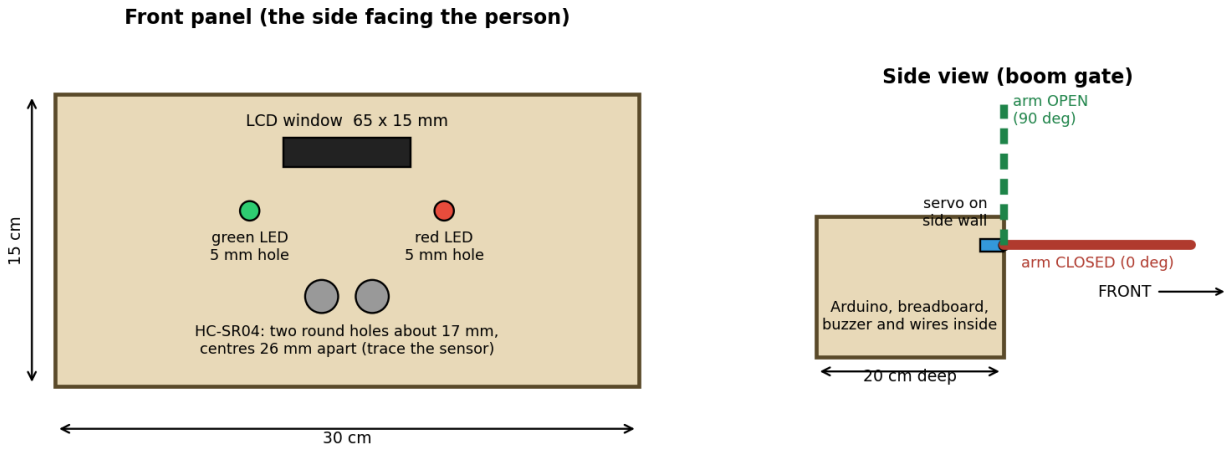
Type	What must happen
`C`	LCD: "Checking PPE". Both LEDs off.
`G`	Green LED on, servo moves to open, one short beep. LCD: "ACCESS GRANTED".
`R`	Red LED on, 3 beeps, servo closed. LCD: "ACCESS DENIED / No helmet+vest".
`H`	Red LED on, 3 beeps. LCD: "Missing: HELMET".
`V`	Red LED on, 3 beeps. LCD: "Missing: VEST".
`I`	Everything off, servo closed. LCD: "PPE Gate / Step closer".

Also move your hand towards the sensor and check that the `D:` number goes down.

If one of these fails, go back to that part's stage and run its small test code again.

Stage 7: Build the physical gate (3-4 hours)

Figure 2: Suggested cardboard gate layout (sizes are suggestions)



Step 1: prepare the box. Choose a sturdy box about 30 cm wide, 15 cm high and 20 cm deep. Close the lid and tape it shut except for one side you can still open (a flap on the back or top) so you can reach the wires. The wide 30 x 15 cm side is the **front panel** and faces the person.

Step 2: mark the front panel. Use a pencil and ruler and follow Figure 2:

- **LCD window:** a rectangle about 65 x 15 mm, near the top centre.
- **Two LED holes:** 5 mm each, about 10 cm apart, below the LCD.
- **Sensor holes:** press the HC-SR04 flat on the cardboard and trace its two round silver cylinders, which are about 17 mm across and 26 mm apart. Put them near the bottom centre.

Step 3: cut the holes. Use a sharp cutter on a cutting mat or thick card (or ask someone to help with the cutting). Cut slowly and keep your fingers away from the blade. Make the sensor holes just big enough for the cylinders to push through.

Step 4: fit the parts to the front panel.

- **LCD:** place it behind the window with its screen facing out, and fix the edges with hot glue or strong double-sided tape. The wires and backpack stay inside the box.
- **Sensor:** push the two cylinders through the holes from the inside, and glue the board's edges to the cardboard.
- **LEDs:** an LED's legs are too short to reach from the panel to the breadboard. Slide the **female end of a male-to-female wire** onto each LED leg, and wrap tape around each join so the legs cannot touch. Push each LED through its hole from the inside, with a drop of glue. Connect the other ends: the wire from the **long leg** goes to the resistor strip (strip 9 for green, strip 17 for red) and the wire from the **short leg** goes to the strip that leads to the - rail (strip 10 for green, strip 18 for red). Take out the LED legs from the breadboard first.

Step 5: buzzer. Leave it on the breadboard, or glue it to the inside wall near a small hole in the cardboard so the sound is not muffled.

Step 6: mount the servo.

1. On the **right side wall**, near the top front corner, cut a rectangular slot about 23 x 13 mm (the size of the servo's body, not including its ears).
2. Push the servo in from **inside** so its shaft (the round spline with the horn) points **outward**. The wider ears stop the servo from going through, so they sit against the inside of the wall.
3. Fix the ears to the wall with the screws that came with it, or with hot glue.

Step 7: fit the arm correctly (important).

1. Upload the Stage 3 test, but change the loop to only `gate.write(0);` so the servo stays at 0 degrees. Leave the USB plugged in.

2. With the servo at 0 degrees, press the white **horn** onto the shaft so that the horn points **horizontally forward** (away from the box front). Fix it with the small screw.
3. Glue or tape the barrier arm (a 20-25 cm ruler, or 3 ice-cream sticks glued together) onto the horn, pointing forward. Decorate it with red and white stripes of tape so it looks like a real barrier.
4. Run the Stage 6 sketch and send `G`: the arm should swing up to point **vertically**. Send `I`: it should go back to horizontal. If the arm goes the wrong way or hits the box, remove the horn and refit it at a different angle.

Step 8: tidy the inside. Fix the Arduino and breadboard to the box floor with double-sided tape or the board's screw holes. Tie loose wires with tape so they cannot pull out. Let the USB cable come out through a small hole in the back.

Step 9: place the gate and the camera.

- Put the gate on a table. Stand the person about **1 to 1.5 m** in front of it.
- Keep the area within about 1 m in front of the sensor **clear of walls, chairs and bags**. Otherwise the sensor will think someone is always standing there.
- The webcam (or laptop) should face the person, with the helmet and vest both in view, at about chest-to-head height if possible. Avoid a bright window directly behind the person. You will check the camera view properly once the software is running.

Step 10: finish. Add a neat title card, for example "PPE Compliance Gate".

---

8. Final hardware acceptance test

Run through this list with the finished gate and tick each line. If every line passes, the hardware is ready for the software.

Check	Pass
Board is detected by the laptop and the sketch uploads	
Serial Monitor shows `D:` values that change when you move your hand	
Reading at 50 cm is close to 50 (within a few cm)	
`C` shows "Checking PPE" on the LCD	
`G`: green LED, servo opens, one beep, LCD "ACCESS GRANTED"	
`R`: red LED, 3 beeps, LCD "No helmet+vest"	
`H` and `V`: correct "Missing" message on the LCD	
`I`: everything off and servo closed	
Barrier arm swings smoothly through its full travel and does not hit anything	
The Arduino does not reset when the servo moves	
Wires are secured and nothing moves when you tap the box	
The USB cable reaches the laptop comfortably	

Write these down for the software stage: the COM port number of the Arduino (for example COM3), and the LCD address you used.

---

9. Troubleshooting summary

Symptom	Likely cause and fix
Upload fails ("port not found")	Wrong board or port selected, bad cable, or the CH340 driver is missing.
Nothing works at all	The - rail is not connected to Arduino GND, or the + rail not to 5V.
LEDs never light	Wrong way round (long leg to the pin side), or missing resistor, or loose wire.
Buzzer silent	Passive buzzer, reversed legs, or loose ground wire.
Servo twitches or the Arduino resets	Add or reverse-check the capacitor, use a powered USB port, or use a separate 5V supply sharing GND.
LCD blank	Turn the contrast screw, check A4 and A5, try address 0x3F, and check the library.
Distance always 999	Trig and Echo are swapped, the VCC or GND wire is loose, or something blocks the sensor.
Everything works for a minute then stops	A loose jumper. Press every wire firmly and re-test.
Servo arm goes the wrong direction	Refit the horn at the 0-degree position as in Step 7.

---

10. Master wiring table (keep this for reference and your report)

Part	Pin or wire	Connects to
Rails	+ rail	Arduino 5V
Rails	- rail	Arduino GND
Capacitor 470 uF	long leg / striped leg	+ rail / - rail
Green LED	long leg (via 220-ohm resistor)	Arduino pin 8
Green LED	short leg	- rail
Red LED	long leg (via 220-ohm resistor)	Arduino pin 9
Red LED	short leg	- rail
Active buzzer	+ (long) leg	Arduino pin 10
Active buzzer	other leg	- rail
Servo	orange / red / brown	Arduino pin 6 / + rail / - rail
I2C LCD	GND / VCC	- rail / + rail
I2C LCD	SDA / SCL	Arduino A4 / A5
HC-SR04	VCC / GND	+ rail / - rail
HC-SR04	Trig / Echo	Arduino pin 4 / pin 5

Serial commands (laptop to Arduino): `C` checking, `G` granted, `R` denied (both missing), `H` denied (helmet missing), `V` denied (vest missing), `I` idle. **Arduino to laptop:** `D:<cm>` about five times a second.

---

11. Safety and care

- Unplug the USB cable before changing wiring.
- Do not power the circuit from anything other than the laptop USB (or a 5V supply for the servo as described).
- Use the cutter carefully, on a cutting mat, and cut away from your hands.
- Keep the gate away from water. Handle the LCD and sensor gently.
- Do not pull wires out by the cable. Hold the plastic end.